

Stoquify POS Enterprise-Grade Audit

Audit date: 2026-08-16
Target: /[locale]/dashboard/pos
Decision: NO-GO for an enterprise production claim
Assessment mode: read-only, evidence-led audit of the current working tree

Executive assessment

Stoquify POS has a materially stronger financial transaction core than its current screen suggests. The final sale path is tenant-scoped, recomputes totals server-side, uses stock and sale compare-and-set guards, records payments and drawer evidence, posts a balanced journal, emits audit and outbox evidence, and returns an explicit receipt retry state after a committed sale. Shift close, refund, void, and receipt-token services also contain credible enterprise controls.

The route is not yet enterprise-ready as an end-to-end operating surface. The primary blockers are contradictory tender contracts, XAF precision acceptance that permits fractional francs, an offline status that is not connected to the checkout path, synthetic hardware readiness, incomplete cashier workflows, draft-cart concurrency gaps, inconsistent entitlement enforcement, an over-privileged seeded cashier, and a responsive layout that makes the cart secondary below the XL breakpoint. Accessibility, privacy, observability, and maintainability gaps compound those risks.

This is therefore a split verdict:

- Financial commit and close controls: strong foundation, conditionally credible.
- Cashier workflow and operational truth: incomplete and sometimes misleading.
- Responsive and accessible usability: below enterprise acceptance.
- Role, entitlement, and seed realism: not acceptable as a least-privilege reference dataset.
- Certification evidence: incomplete because current authenticated browser inspection and opt-in PostgreSQL tests remain unexecuted.

Readiness scorecard

Dimension	Rating	Decision basis
Financial integrity	4/5	Server recalculation, transactional posting, CAS inventory guards, audit/outbox evidence
Core cashier workflow	2/5	Park and override are placeholders; refund/void are absent from this screen
Payments	2/5	Mobile Money cannot satisfy its service contract from the UI; Store Credit is selectable but always rejected
Shift and cash controls	3/5	Strong close transaction, but XAF fractional input and no explicit inactivity/handover flow
Offline resilience	1/5	Queue hooks exist, but checkout does not consume them; status language overstates readiness

Dimension	Rating	Decision basis
Hardware integration	1/5	Scanner, printer, drawer, and network readiness are inferred or hard-coded
Access and privacy	2/5	Strong action checks in places; route/navigation/module policy and seeded role scope conflict
Accessibility	2/5	Keyboard shortcuts exist; accessible names, live status, touch targets, focus and zoom evidence are incomplete
Responsive usability	2/5	Cart becomes a long-scroll secondary region below XL
Performance and scale	2/5	Undebounced query keys, bounded but unpaginated catalog, broad invalidation, large client component
Localization	3/5	Main POS EN/FR parity is exact; offline strip has local copy and weak French typography
Test and release evidence	3/5	199 expanded POS tests pass and typecheck passes; browser matrix and PostgreSQL certification are incomplete

Scope, method, and evidence limits

The audit covered the page route, route guards, sidebar registration, the principal POS client component, POS hooks and actions, POS services and schemas, payment reconciliation, offline hooks/status, seed role construction, tests, EN/FR messages, recent UI/UX evidence, and graphify architecture outputs.

Runtime verification was deliberately non-mutating. Before the browser interruption, the unauthenticated browser reached `/en/login` after requesting `/en/dashboard/pos`, confirming the authentication boundary. The resumed run authorized seeded-cashier login, but the new browser first reached `localhost` while the development server was offline and then blocked subsequent `localhost` navigation under its URL safety policy. That control was not bypassed. No cart line, shift, tender, refund, void, receipt token, database row, migration, or seed file was changed. Historical screenshots from 2026-08-15 were used only to corroborate layout observations, not as current cashier proof.

Evidence register: `what-next/evidence/pos-enterprise-grade-audit-2026-08-16/README.md`.

Architecture and dependency trace

The current implementation concentrates the cashier surface in `components/pos/ProfessionalPOSSystem.tsx`, a 2,493-line client component. It owns location and terminal selection, shift dialogs, catalog search, cart editing, tender construction, receipt preferences, customer selection, recent/favorite items, hardware labels, and last-sale evidence. `hooks/posHooks/usePosOperations.ts` is the React Query bridge, while `services/pos/pos.service.ts` contains 3,262 lines spanning session, cart, sale, close, refund, and void behavior.

The checked graph topology reinforces the coupling:

- `graph_components.json` places `ProfessionalPOSSystem` and its local operations in high-connectivity community 1; its test surface is separated in community 45.
- `graph_hooks.json` places `usePosOperations` and eighteen POS operations in community 1, while offline behavior sits in community 20.
- `graph_actions.json` separates catalog, cart, and session work into communities 67, 78, and 97, exposing a backend decomposition that the UI does not mirror.

- graph_app.json places the POS page in community 140 and its error boundary in community 139.

Graph line metadata may be stale relative to the dirty working tree, so it is used for relationship evidence, not exact line attribution.

Current cashier workflow

```

flowchart LR
    A["Authenticated user"] --> B["Route guard: OPERATE_POS"]
    B --> C["Select location and terminal"]
    C --> D["Active shift?"]
    D -- "No" --> E["Enter opening float and open shift"]
    D -- "Yes" --> F["Search or browse catalog"]
    E --> F
    F --> G["Add or update draft cart"]
    G --> H["Choose customer and receipt"]
    H --> I["Build one or more tenders"]
    I --> J["Client preview valid?"]
    J -- "No" --> I
    J -- "Yes" --> K["Server commit transaction"]
    K --> L["Sale, stock, payment, drawer, journal, audit, outbox"]
    L --> M["Receipt delivery"]
    M -- "Success" --> N["Completed"]
    M -- "Retry required" --> O["Committed sale with recoverable receipt state"]

```

Critical breaks in this flow are the tender metadata mismatch, selectable unsupported Store Credit, divergent available/on-hand checks, draft write races, and missing real offline/hardware states.

Controls that are already credible

- app/[locale]/(dashboard)/dashboard/pos/page.tsx:16 enters through withPosSurfaceAccess and requires OPERATE_POS.
- services/pos/pos.service.ts scopes sale commit to organization, location, terminal, session, user, and active state; recomputes financial values instead of trusting the browser.
- Inventory issuance uses quantity and version guards, and the sale state is finalized with a compare-and-set update.
- Provider references are normalized and checked for duplicate capture where required.
- Cash sales record drawer evidence; ON_ACCOUNT sales enforce customer and credit-limit rules.
- Sale commit writes accounting, audit, event, and outbox records in the transaction and distinguishes post-commit receipt retry from financial rollback.
- Shift close uses a serializable transaction, stable idempotency key, evidence replay validation, drawer/terminal/session transitions, and an audit/event trail.
- Refund and void actions use sensitive permission checks and a 300-second fresh-auth window at actions/pos/tender.actions.ts:70 and :89.
- Public receipt tokens use a hash-backed registry and redacted public data; token revocation uses fresh authentication at actions/pos/receipt-token.actions.ts:116.
- Main pos message namespaces have 245 English and 245 French keys with no missing counterpart.

These strengths are necessary but do not remove the workflow and control contradictions below.

Prioritized findings

ID	Severity	Finding	Primary owner	Estimated effort
POS-01	P1	Mobile Money and Store Credit UI contracts contradict service behavior	Payments + POS	M
POS-02	P1	XAF inputs accept and persist fractional francs inconsistently	POS + Money platform	M
POS-03	P1	Offline readiness is displayed without checkout integration	POS + Offline platform	L
POS-04	P1	Hardware readiness is inferred or hard-coded	POS + Device platform	L
POS-05	P1	Seeded cashier violates least privilege	Identity + Seed data	S
POS-06	P1	POS entitlement is inconsistent across navigation, route, reads, and writes	Identity + Modules	M
POS-07	P1	Draft cart creation and line mutation lack enterprise concurrency contracts	POS backend	M
POS-08	P1	Inventory availability is interpreted inconsistently before commit	Inventory + POS	M
POS-09	P1	Park, manager override, refund, void, handover, and recovery workflows are incomplete	Product + POS	L
POS-10	P1	Tablet/mobile layout demotes the cart and tender below a large catalog	POS UI	M
POS-11	P1	Accessibility and touch-operation evidence is insufficient	Design system + POS	M
POS-12	P1	No POS-specific inactivity lock and cashier handover state	Identity + POS	M
POS-13	P2	Customer and receipt data are more exposed than necessary	Privacy + POS	S
POS-14	P2	Search, invalidation, images, and catalog loading do not meet a clear performance budget	POS UI + API	M
POS-15	P2	A 2,493-line screen and 3,262-line service create high change risk	POS engineering	L
POS-16	P2	Offline/local French copy bypasses the main translation contract	Localization + POS	S
POS-17	P2	Visual hierarchy is dense and readiness states dominate selling tasks	Product design	M
POS-18	P2	Operational observability and certification coverage remain incomplete	SRE + QA	M

P0 was not assigned because the audit did not reproduce active corruption, tenant leakage, or a committed double sale. P1 means the issue blocks an enterprise-ready claim or a controlled pilot at meaningful scale.

Finding packets

POS-01 — Tender options contradict the payment contract

- Workflow/state: tender construction and sale commit.
- Evidence: ProfessionalPOSSystem.tsx:81, :166, :276, and :299 model tender lines with method, amount, and optional reference only; the selector offers MOBILE_MONEY and STORE_CREDIT. payment-reconciliation.service.ts:157-171 requires both a Mobile Money provider reference and mobileMoneyProvider. pos.service.ts:89-95 rejects every STORE_CREDIT tender until a liability ledger is implemented.
- Current behavior: a cashier can select methods that the current UI cannot complete successfully.
- Why insufficient: a primary Cameroon tender must be deterministic before the charge action. A selectable method is an operational promise.
- User/business/control impact: avoidable queue delay, cashier confusion, repeated charge attempts, and risk of an external provider charge without a completed internal sale if staff work around an error.
- Root cause: UI tender DTO and capability catalog are not generated from the authoritative payment-method contract.
- Severity: P1.
- Modification: return enabled tender capabilities per tenant/location/terminal. Mobile Money must require provider selection plus transaction reference; Store Credit must be hidden or disabled with an explicit reason until the ledger exists. Validate per method in UI and service from one schema.
- Layer/owner: POS UI, payment domain, module entitlement; Payments + POS owners.
- Dependencies/effort: provider configuration, translation, reconciliation; M.
- Acceptance criteria: no selectable tender lacks all required fields; unsupported methods cannot reach commit; error copy is field-specific; EN/FR parity exists.
- Verification: contract tests for every method, browser tests for single/split tender, duplicate reference tests, and a sandbox provider reconciliation test.

POS-02 — XAF permits fractional cash values that its presentation later rounds away

- Workflow/state: open shift, tender amount, close shift, and variance review.
- Evidence: ProfessionalPOSSystem.tsx:1369 and :2051 use step 0.01. pos.schemas.ts:45 accepts the generic explicit closing-count schema. pos.service.ts:1266 rounds actualBalance to two decimals, while :593-594 derives variance precision from currencyFractionDigits. XAF tests explicitly demonstrate that sub-franc differences produce no visible variance warning.
- Current behavior: fractional XAF can enter the model, while variance calculation and display operate at zero fraction digits.
- Why insufficient: persisted cash evidence and displayed cash truth can disagree.
- Impact: unexplained drawer drift, misleading zero variance, reconciliation noise, and fragile audit evidence.
- Root cause: generic two-decimal money input is used before tenant currency precision is applied.
- Severity: P1.

- Modification: expose authoritative currency scale in the POS contract; normalize and reject non-representable amounts server-side; use scale-aware inputMode, step, formatting, tender preview, and variance thresholds. Migrate or quarantine any fractional XAF evidence.
- Layer/owner: money primitives, schemas, POS service/UI; POS + Money platform.
- Dependencies/effort: organization currency and historical-data check; M.
- Acceptance criteria: XAF accepts integers only; decimal currencies retain their valid scale; stored, posted, and displayed values are identical; no silent rounding.
- Verification: property tests by currency, schema tests, UI keyboard tests, database assertions, and close replay tests.

POS-03 — Offline status overstates a disconnected capability

- Workflow/state: network loss, queue, replay, conflict, and operator recovery.
- Evidence: OfflineSyncStatusStrip.tsx:25 says "Offline degraded path armed." useOfflineSync.ts:176 and :196 export enqueue and flush hooks, but repository search found no checkout consumer. ProfessionalPOSSystem imports the status strip, not the enqueue/flush mutations; normal add/update/commit operations call live server actions.
- Current behavior: the UI can imply offline preparedness although checkout is not routed through the offline state machine.
- Why insufficient: an enterprise cashier must know whether a sale is local-only, queued, replaying, conflicted, rejected, or durably posted.
- Impact: false confidence during outages, duplicate attempts, uncertain stock/payment state, and unverifiable recovery.
- Root cause: monitoring UI was shipped separately from workflow integration and certification.
- Severity: P1.
- Modification: until certified, label the strip "Online checkout required" and block commits on loss. For actual offline mode, introduce signed/idempotent commands, local encrypted queue, monotonic terminal sequence, explicit provisional receipts, stock/payment policy, conflict workbench, deterministic replay, and purge/retention controls.
- Layer/owner: POS client, sync actions, local storage, service idempotency, operations; POS + Offline platform.
- Dependencies/effort: service worker/device policy, identity expiry, receipt semantics; L.
- Acceptance criteria: every state is truthful; replay is idempotent; duplicate commands yield one sale; conflict/rejection is actionable; logout/terminal handover protects queued data.
- Verification: offline browser suites, kill/restart tests, duplicate replay, expired-session replay, stock conflict, provider timeout, and telemetry assertions.

POS-04 — Hardware readiness is synthetic

- Workflow/state: scan, print, drawer, and connectivity readiness.
- Evidence: ProfessionalPOSSystem.tsx:663-667 derives scanner readiness from canSell, drawer readiness from an open database drawer, and sets printer and network ready to true. The shell announces scanner readiness at :1191.
- Current behavior: business state is presented as device health.
- Why insufficient: an open logical drawer is not proof that a physical drawer, scanner, or printer is connected and responding.

- Impact: failed scans/receipts, unattended cash-drawer problems, longer queues, and misleading operational telemetry.
- Root cause: placeholder readiness objects are rendered as authoritative status.
- Severity: P1.
- Modification: define a device-adapter contract with connected, ready, degraded, disconnected, unknown, stale, last-seen, test action, and recoverable error. Default to unknown; never infer device health from sale permission.
- Layer/owner: device integration, POS UI, telemetry; Device platform + POS.
- Dependencies/effort: supported hardware matrix and kiosk/runtime bridge; L.
- Acceptance criteria: status comes from adapters with freshness; print/scanner/drawer failures offer retry/fallback; unsupported devices show unknown or unavailable.
- Verification: adapter contract tests, simulated disconnect/reconnect, stale heartbeat, browser UI state tests, and store hardware pilot.

POS-05 — The reference cashier is over-privileged

- Workflow/state: role provisioning and least-privilege validation.
- Evidence: comprehensive-seed.ts:475 defines permissionsContaining over all permission strings. The cashier block at :590-595 adds every permission containing POS, CASH, PAYMENT, or RECEIPT. Inspection of the generated credential artifact found unrelated permissions including POST_JOURNAL_ENTRIES and SUPPLIER_PAYMENTS_MANAGE. No password is reproduced in this report.
- Current behavior: the synthetic cashier can receive materially broader authority than the intended role.
- Why insufficient: a realistic enterprise seed is executable access-control documentation and should demonstrate separation of duties.
- Impact: misleading demos/tests, hidden authorization regressions, and unsafe reuse of the role recipe.
- Root cause: substring-based permission construction.
- Severity: P1.
- Modification: replace the helper with an explicit cashier allowlist from the canonical role policy. Add a denylist assertion for accounting posting, supplier payments, payroll, administration, role management, and protected refund/void permissions.
- Layer/owner: seed data and RBAC configuration; Identity + Seed data.
- Dependencies/effort: authoritative cashier role matrix; S.
- Acceptance criteria: seeded cashier has only documented POS permissions; negative permission tests fail on any future expansion; seed catalog publishes the allowlist.
- Verification: snapshot and denied-action tests using the seeded account.

POS-06 — Entitlement behavior differs by entry point

- Workflow/state: navigation, route access, reads, and financial writes.
- Evidence: sidebar.ts:292 registers the section under moduleSlug sales and :299 gives the POS child only OPERATE_POS. pos-route-data-access.ts:10 explicitly has module?: never. Catalog actions observe moduleSlug pos, while tender, refund/void, and receipt-token actions enforce pos.
- Current behavior: a user can see or enter the route under Sales/permission logic but hit different module policy during reads or writes.

- Why insufficient: enterprise access must fail consistently before work begins and must explain whether permission, subscription, module, location, or terminal assignment is missing.
- Impact: broken checkout after cart preparation, support tickets, leakage of unentitled reads in observe mode, and non-deterministic package behavior.
- Root cause: no single POS surface-access contract shared by sidebar, route, server actions, and services.
- Severity: P1.
- Modification: define one POS entitlement descriptor and use it for navigation visibility, route guard, read actions, mutations, receipts, offline sync, and tests. Choose one module slug and one rollout mode per environment.
- Layer/owner: route policy, modules, actions, navigation; Identity + Modules.
- Dependencies/effort: package strategy and migration window; M.
- Acceptance criteria: the same principal gets the same allow/deny outcome at every boundary, with a safe reason code; privileged actions still layer fresh auth.
- Verification: permission × entitlement × location matrix tests and browser denied-state tests.

POS-07 — Draft carts lack a concurrency and idempotency boundary

- Workflow/state: create draft, add/update/remove line, resume cart.
- Evidence: pos.service.ts:828-887 performs find-then-create for a draft order. Add line performs another find-then-update/create at :1672-1688. The schema has no demonstrated unique invariant for one active draft per cashier/session/terminal and no unique salesOrderId + itemId line invariant. Cart mutations do not carry commandId or expected cart revision.
- Current behavior: concurrent tabs, repeated scans, retries, or double clicks can race before the strongly protected final commit.
- Why insufficient: draft correctness is part of payment correctness; duplicate lines or carts can alter what the cashier believes will be charged.
- Impact: duplicate quantities, orphan drafts, last-write-wins edits, and reconciliation/support burden.
- Root cause: optimistic client mutation without a database-enforced aggregate identity and version.
- Severity: P1.
- Modification: add a partial uniqueness strategy for active draft identity, unique line identity, cart version, and idempotent mutation command table/key. Return authoritative cart plus conflict metadata after every command.
- Layer/owner: Prisma schema, POS service, hooks; POS backend.
- Dependencies/effort: migration and draft cleanup; M.
- Acceptance criteria: repeated identical command produces one effect; concurrent incompatible changes return a conflict; only one active draft exists per defined scope.
- Verification: PostgreSQL race tests with two clients, command replay, double-scan, and stale-version browser tests.

POS-08 — Pre-commit stock rules switch between available and on-hand

- Workflow/state: catalog, add line, update quantity, and commit.
- Evidence: pos.service.ts:1602 and :1664-1688 use quantityAvailable for catalog/add. Update at :1770-1774 clamps against quantityOnHand. The UI computes additional cart availability from on-hand while separately displaying available. Final issuance correctly guards both on-hand and available.

- Current behavior: a quantity can appear acceptable during editing and then fail only at final commit when reservations reduce available stock.
- Why insufficient: one authoritative sellable quantity and one conflict policy must govern the entire workflow.
- Impact: end-of-queue failures, oversell attempts, cashier workarounds, and poor customer experience.
- Root cause: read-model and mutation code use different inventory concepts.
- Severity: P1.
- Modification: define sellableAvailable as the POS contract; return inventory version and reservation reason; validate add/update against it; refresh or resolve conflict before payment; retain final CAS as the last defense.
- Layer/owner: inventory read model, POS service/UI; Inventory + POS.
- Dependencies/effort: reservation policy and stock freshness SLA; M.
- Acceptance criteria: displayed maximum equals server preflight maximum; conflicts identify item and new available quantity; no silent clamping.
- Verification: reserved-stock integration tests, concurrent sale tests, and stale-cart browser tests.

POS-09 — Required exception workflows are placeholders or absent

- Workflow/state: hold/resume, manager override, refund, void, cancellation, handover, and recovery.
- Evidence: ProfessionalPOSSystem.tsx:1160 defines handleBackendGatedFeature; Park sale and Manager override call it at :1432 and :1436. Refund and void actions/services exist with strong controls but are not imported or surfaced on this page. No complete handover or suspended-cart recovery flow is presented.
- Current behavior: happy-path checkout is prominent; exception commands terminate in a toast or live elsewhere without a coherent cashier journey.
- Why insufficient: enterprise POS quality is determined by exception handling under queue pressure.
- Impact: abandoned carts, supervisor password sharing, manual refund workarounds, and loss of traceability.
- Root cause: backend capabilities and UI workflow design evolved separately.
- Severity: P1.
- Modification: replace placeholders with explicit disabled capability cards until implemented, then build reason-coded commands, manager approval handoff, receipt/order lookup, partial/full refund, void eligibility, hold ownership/expiry, suspended-session recovery, and handover reconciliation.
- Layer/owner: Product, POS UI/actions/services, identity/audit; Product + POS.
- Dependencies/effort: approval policy and state diagrams; L.
- Acceptance criteria: every visible command completes, is honestly disabled, or links to an authorized workflow; every exception records actor, approver, reason, before/after, and correlation ID.
- Verification: role-separated end-to-end tests for allowed, denied, expired-auth, replay, and recovery paths.

POS-10 — The responsive layout puts checkout after the catalog below XL

- Workflow/state: product discovery, cart review, tender, and completion on tablet/mobile.
- Evidence: ProfessionalPOSSystem.tsx:1234 becomes a two-column catalog/cart layout only at xl, with a fixed 420px cart. Before that breakpoint, DOM order places catalog panels and up to dozens of product cards before the cart. Historical 2026-08-15 EN/FR screenshots corroborate the dense pre-cart stack but are not current proof.
- Current behavior: desktop at XL can sustain dual-pane operation; tablet and mobile make the cart/tender a downstream scroll destination.

- Why insufficient: cart total, blockers, and charge action must remain continuously reachable during a high-throughput task.
- Impact: slower checkout, missed cart errors, accidental duplicate adds, and poor one-handed tablet use.
- Root cause: responsive collapse follows component DOM order rather than cashier task priority.
- Severity: P1.
- Modification: use a persistent cart rail on desktop, split-pane or sticky cart summary on tablet, and bottom-sheet cart/charge pattern on mobile. Keep a visible item count, total, blocker, and "Review/Charge" affordance at every size.
- Layer/owner: POS information architecture and CSS; POS UI.
- Dependencies/effort: design-system responsive primitives; M.
- Acceptance criteria: at 390×844, 768×1024, 1024×768, and 1440×1000, cart total and next action remain reachable without traversing the catalog; no overlap at 200% zoom.
- Verification: EN/FR screenshot baselines, Playwright task-time tests, orientation changes, virtual keyboard, and touch-only walkthrough.

POS-11 — Accessibility and touch operation are not release-proven

- Workflow/state: catalog controls, cart edits, status feedback, dialogs, and keyboard operation.
- Evidence: several icon-only controls rely on title, including the tender remove action at ProfessionalPOSSystem.tsx:2063-2065; grid/list and favorite/detail controls lack a consistent aria-label/aria-pressed contract. Standard quantity controls are approximately 28px unless touch mode is enabled. OfflineSyncStatusStrip has no demonstrated role=status or aria-live. Keyboard shortcuts exist, including search, Escape, F4, and F8, but the shortcut footer is hidden at small sizes.
- Current behavior: some keyboard acceleration exists, while accessible naming, state announcement, focus, target size, and zoom behavior are inconsistent or unverified.
- Why insufficient: a cashier must operate quickly with keyboard, touch, low vision, screen reader, motor limitation, or noisy hardware feedback.
- Impact: exclusion, slower queues, higher error rate, and WCAG 2.2 risk.
- Root cause: local controls do not consistently inherit an enterprise POS accessibility contract.
- Severity: P1.
- Modification: require accessible names and pressed/selected semantics; minimum 44×44 touch targets; cart/status live regions that avoid chatter; visible focus; deterministic dialog focus restore; reduced-motion support; no color-only state; documented shortcut discoverability and conflict handling.
- Layer/owner: design system and POS UI; Accessibility + POS.
- Dependencies/effort: component primitives and test harness; M.
- Acceptance criteria: WCAG 2.2 AA automated checks pass; full checkout is keyboard-only; screen reader announces cart delta, blockers, commit result, and offline/device state; 200% zoom does not lose functionality.
- Verification: axe, NVDA/Chrome, keyboard script, touch target audit, contrast, zoom, and reduced-motion tests in EN/FR.

POS-12 — Inactivity lock and cashier handover are missing from the POS contract

- Workflow/state: unattended terminal, break, cashier replacement, and shift recovery.

- Evidence: POSSession models active/suspended/closed/reconciled lifecycle behavior, but the inspected POS surface has no last-activity lock, lock owner, supervisor unlock, or explicit handover screen. Active shift polling is periodic and general application auth does not prove a cashier-terminal lock policy.
- Current behavior: authorization is checked at requests, but unattended-screen and operator continuity are not visibly controlled.
- Why insufficient: shared physical terminals need a shorter, auditable operational lock than a general web session.
- Impact: unauthorized sales/customer exposure, misattributed drawer activity, and weak accountability.
- Root cause: web-session authentication is being treated as equivalent to terminal custody.
- Severity: P1.
- Modification: add configurable inactivity lock, quick re-auth, terminal/session ownership display, suspend/handover command, pending-cart policy, queued-offline policy, and supervisor recovery with reason.
- Layer/owner: auth/session, POS service/UI, audit; Identity + POS.
- Dependencies/effort: policy values, PIN/passkey decision, offline semantics; M.
- Acceptance criteria: a locked terminal reveals no customer/cart detail; unlock rechecks membership and session ownership; handover cannot silently inherit another cashier's cash responsibility.
- Verification: timer, multi-user, stale membership, browser refresh, offline, and audit attribution tests.

POS-13 — Customer and receipt information exceeds point-of-action need

- Workflow/state: customer lookup, last sale, and receipt-token management.
- Evidence: customer rows include full contact fields plus order count and totalRevenue at ProfessionalPOSSystem.tsx:2466. The last-sale panel prints digitalReceiptUrl at :1994-1996. Receipt-token history is embedded in the tender area when capability allows.
- Current behavior: a cashier can see commercial history and raw delivery/token-related values in the high-traffic selling surface.
- Why insufficient: POS data should be minimized for the task and protected from shoulder surfing.
- Impact: customer privacy exposure and unnecessary disclosure of internal receipt links or spending history.
- Root cause: the customer and receipt DTOs are reused without a cashier-minimized projection.
- Severity: P2.
- Modification: provide a POS-specific masked customer projection; hide revenue by default; render receipt actions, not raw URLs; move token administration to a privileged secondary view; redact logs and screenshots.
- Layer/owner: read models, privacy, UI; Privacy + POS.
- Dependencies/effort: field-purpose matrix; S.
- Acceptance criteria: cashier responses contain only permitted fields; links are opaque actions; managers receive expanded data only under explicit permission.
- Verification: DTO contract tests, role snapshots, browser masking tests, log scan, and public-token redaction tests.

POS-14 — Scale behavior has no enforceable performance budget

- Workflow/state: search, product browsing, mutation refresh, image loading, and polling.

- Evidence: search text participates directly in query keys without a demonstrated debounce. Catalog requests are bounded but not cursor-paginated. usePosOperations.ts:152, :179, :190, :201, :225, and :236 broadly invalidate posOperationsKeys.all after mutations. Product images have no demonstrated explicit lazy/error strategy. The active shift is polled.
- Current behavior: small seeded catalogs are workable; larger catalogs can generate request churn and broad refetches.
- Why insufficient: enterprise readiness needs measured p95/p99 budgets at realistic item, branch, and concurrency counts.
- Impact: scanner lag, search flicker, network cost, and long checkout times in weaker store networks.
- Root cause: convenient query invalidation and monolithic rendering without workload targets.
- Severity: P2.
- Modification: scan-first exact lookup path; 150–250ms text debounce; cursor pagination/virtualization; precise cache updates; image lazy loading/fallback; visibility-aware polling; server indexes verified against explain plans.
- Layer/owner: UI, React Query, API, database; POS UI + API.
- Dependencies/effort: production-like data generator and telemetry; M.
- Acceptance criteria: define and meet budgets such as scan-to-cart p95 ≤ 300 ms online, local interaction ≤ 100 ms, search p95 ≤ 500 ms, commit p95 ≤ 2 s excluding external provider time, and no unbounded DOM growth.
- Verification: k6/API load, browser traces, React profiling, query-count assertions, and database explain plans.

POS-15 — The component and service boundaries are too large to change safely

- Workflow/state: all POS states.
- Evidence: ProfessionalPOSSystem.tsx is 2,493 lines and holds a large cross-cutting state set. pos.service.ts is 3,262 lines. Graph communities show backend concepts separated but client orchestration concentrated around one high-degree component.
- Current behavior: a change to tender, shift, catalog, customer, receipt, or device presentation frequently touches the same component; multiple domain writes share one service module.
- Why insufficient: financial UI evolution needs reviewable boundaries, local tests, and explicit state machines.
- Impact: regression risk, merge conflicts, slower review, and harder ownership.
- Root cause: incremental feature accumulation in one client controller and one service file.
- Severity: P2.
- Modification: extract workflow shells by state rather than cosmetic fragments: TerminalBootstrap, ShiftLifecycle, CatalogSearch, CartEditor, TenderPanel, ReceiptOutcome, and ExceptionActions. Split backend commands from reads and refund/void/close domains. Preserve public contracts during extraction.
- Layer/owner: POS frontend/backend architecture; POS engineering.
- Dependencies/effort: characterization tests and staged refactor; L.
- Acceptance criteria: each workflow has a typed state contract and focused tests; no duplicate business rules move client-side; file boundaries reduce coupling without changing behavior.
- Verification: existing regression suites, new component contracts, graph delta, and bundle analysis.

POS-16 — Offline status copy bypasses localization governance

- Workflow/state: sync status and retry.

- Evidence: OfflineSyncStatusStrip.tsx defines English and French strings locally. French includes unaccented forms such as etat, acces, certifie, degrade, Cloture, enrole, and Reessayer. These strings are outside the otherwise parity-complete pos namespace.
- Current behavior: the main page has key parity, but a critical reliability control uses separate copy and inconsistent typography.
- Why insufficient: reliability states must be precise and equally trustworthy in both supported languages.
- Impact: reduced comprehension and translation drift during incident states.
- Root cause: component-local dictionary rather than the centralized message catalog.
- Severity: P2.
- Modification: move all offline/device/recovery copy into messages/en.json and messages/fr.json; use professional French review; add interpolation and length tests.
- Layer/owner: localization resources and offline UI; Localization + POS.
- Dependencies/effort: none material; S.
- Acceptance criteria: no user-facing literal remains in the strip; parity and pseudo-localization pass; labels remain usable at narrow widths.
- Verification: static literal scan, locale tests, FR reviewer signoff, and responsive screenshots.

POS-17 — Visual hierarchy prioritizes status density over selling

- Workflow/state: initial page scan and repeated product-to-cart cycle.
- Evidence: the component forces a dark presentation and stacks terminal/shift context, smart actions, device tiles, performance metrics, filters, categories, product cards, cart, tender, last-sale details, and shortcut help. Metric labels use text-[10px] at ProfessionalPOSSystem.tsx:343. Product cards carry image, name, SKU/barcode/category/brand, three stock metrics, and progress treatment.
- Current behavior: information is plentiful but the primary next action competes with operational metadata.
- Why insufficient: a professional POS optimizes glanceability, strong totals, error salience, and low cognitive switching in bright, noisy environments.
- Impact: slower visual parsing, fatigue, and missed blockers.
- Root cause: dashboard information density applied to a transactional workstation.
- Severity: P2.
- Modification: adopt three levels: persistent terminal/shift truth, dominant catalog/cart/charge task, and collapsible diagnostics. Increase minimum text size, simplify stock semantics, preserve high-contrast light and dark modes, and reserve strong color for state/action.
- Layer/owner: product design and POS UI; Product design.
- Dependencies/effort: POS visual tokens and store usability study; M.
- Acceptance criteria: first-time and trained users identify terminal, shift, cart total, blocker, and next action in under five seconds; sunlight/contrast and dense-catalog studies pass.
- Verification: moderated cashier study, visual regression, contrast measurement, and task-time benchmark.

POS-18 — Observability and release certification are incomplete

- Workflow/state: search, scan, cart mutation, commit, provider capture, receipt delivery, close, sync, and exceptions.

- Evidence: domain audit/events are strong, but repository inspection found no complete POS-specific latency/error/queue/device measurement contract. The expanded selection passed 199 tests with seven skipped; the opt-in PostgreSQL close suite remains skipped. The authenticated browser matrix is blocked by the recorded localhost browser-policy state.
- Current behavior: failures can often be audited after the fact, but operators and SREs lack a demonstrated real-time POS health model and release gate.
- Why insufficient: enterprise operations require detection, correlation, alerting, and reproducible certification across financial and human workflows.
- Impact: slow incident isolation, unknown store blast radius, and unsupported readiness claims.
- Root cause: transaction evidence, product analytics, and release evidence are not unified.
- Severity: P2.
- Modification: define correlation IDs from browser command through transaction/outbox; structured error taxonomy; metrics for scan/search/cart/commit/provider/receipt/close/offline/device; store/tenant-safe dashboards; redaction; SLOs; synthetic checkout without financial persistence; mandatory Postgres and browser gates.
- Layer/owner: client/server instrumentation, SRE, QA; SRE + QA.
- Dependencies/effort: telemetry platform, safe synthetic tenant; M.
- Acceptance criteria: an operator can identify affected store, terminal, workflow phase, safe retry status, and correlation ID without sensitive payloads; release gate links immutable results.
- Verification: injected failures, alert tests, trace continuity, redaction review, and release-pipeline evidence.

Reviewer-lens decisions

Reviewer lens	Decision	Evidence-backed conclusion
Enterprise/platform architecture	Implemented but incomplete	Backend action communities are separated, but the POS client and main service remain high-coupling hubs; POS-15.
POS backend and integrations	Confirmed defects/gaps	Tender capability and payment-capture contracts conflict; provider and device integration states are incomplete; POS-01 and POS-04.
Database and transaction integrity	Implemented but incomplete	Final commit and close controls are strong; draft uniqueness, mutation idempotency, and XAF normalization remain deficient; POS-02 and POS-07.
Security, tenant isolation, RBAC, privacy, fraud	Implemented but incomplete	Tenant/action checks and fresh auth are credible, while seed privilege, entitlement consistency, terminal custody, and minimization need remediation; POS-05, POS-06, POS-12, POS-13.
Frontend and design system	Below target	Responsive task priority, control semantics, information density, and component size prevent an enterprise UI claim; POS-10, POS-11, POS-15, POS-17.
Workflow and service UX	Missing required paths	Happy-path checkout exists; park, override, refund/void entry, handover, and recovery are not coherent on the page; POS-09.
Accessibility	Blocked by missing evidence and code gaps	No WCAG claim is supportable; missing semantics/touch/live-region evidence is material; POS-11.
Localization and content design	Implemented but incomplete	Main POS EN/FR key parity is exact; offline/local reliability copy bypasses governance; POS-16.

Reviewer lens	Decision	Evidence-backed conclusion
Retail operations, inventory, cash control, reconciliation	Implemented but incomplete	Cash close and final inventory controls are strong, while pre-commit availability, device truth, fractional XAF, and exception operations are weak; POS-02, POS-04, POS-08, POS-09.
Offline/edge and distributed consistency	Missing required capability if advertised	Queue infrastructure exists but is not integrated into sale execution; POS-03.
Product strategy and business process	Below target	The product exposes unsupported or placeholder capabilities and makes diagnostics compete with selling; POS-01, POS-09, POS-17.
Quality and release assurance	Blocked by missing evidence	Unit/integration selection is green, but current browser matrix and opt-in PostgreSQL certification are incomplete; POS-18.
SRE, resilience, performance, cost	Implemented but incomplete	Domain evidence exists, but no enforceable POS SLO/telemetry/performance gate was found; POS-14 and POS-18.
SaaS entitlement, packaging, adoption, support	Confirmed inconsistency	Sales navigation, permission route, observed reads, and enforced POS writes do not share one package contract; POS-06.
OHADA/SYSCOHADA statutory and country-pack compliance	Not applicable to interface certification	The audit verifies that sale posting has accounting provenance, but no statutory pack, tax configuration, journal mapping, or expert-reviewed country evidence was certified here.
Analytics and data governance	Implemented but incomplete	Operational and audit events exist, but a governed, minimized POS telemetry model is not demonstrated; POS-13 and POS-18.
AI/agent safety and human approval governance	Not applicable	No AI-generated decision or autonomous POS action is present in the inspected cashier workflow. Supervisor approval remains a conventional workflow requirement under POS-09.
Billing and growth	Not directly applicable	Commercial entitlement consistency is relevant under POS-06; pricing, billing conversion, and growth experiments are outside this narrow transactional audit.

Truth classification

Classification	Items
Already implemented and satisfactory	Tenant-scoped final sale transaction; server-owned totals; inventory CAS at commit; drawer/payment/journal/audit/outbox evidence; serializable shift close; sensitive refund/void and receipt-token fresh auth; main POS EN/FR key parity
Implemented but incomplete	Route access, tender UI, inventory preflight, shift lifecycle, receipt recovery UI, device status, telemetry, responsive behavior, accessibility, privacy minimization, modular boundaries
Confirmed defect	Unsupported Store Credit remains selectable; Mobile Money provider cannot be supplied; fractional XAF can be accepted then hidden by zero-decimal variance; synthetic hardware readiness; seeded cashier substring privilege expansion
Missing but required	Draft idempotency/version contract, inactivity lock/handover, complete exception workflows, authoritative device state, current browser release evidence, consistent entitlement gate

Classification	Items
Optional enhancement	After core remediation: customer-facing display, configurable quick keys, role-safe sales recommendations, device fleet dashboard, evidence-backed predictive replenishment prompts
Not applicable	AI autonomous-action governance and a statutory compliance certification decision for this UI-only audit
Blocked by missing evidence	Authenticated EN/FR desktop/tablet/mobile inspection; populated-cart/tender/error runtime states; opt-in PostgreSQL close suite; physical hardware and degraded-network certification

Target-state workflow

The target POS should expose one unambiguous state machine:

```
stateDiagram-v2
    [*] --> AccessCheck
    AccessCheck --> Blocked: permission, entitlement, assignment, or auth failure
    AccessCheck --> TerminalReady: authorized
    TerminalReady --> ShiftRequired: no owned active shift
    TerminalReady --> Selling: owned active shift
    ShiftRequired --> Selling: server-confirmed open
    Selling --> CartConflict: stale version or stock change
    CartConflict --> Selling: authoritative cart accepted
    Selling --> Tendering: cart passes server preflight
    Tendering --> ApprovalPending: controlled exception requires supervisor
    ApprovalPending --> Tendering: approval granted and fresh
    Tendering --> Committing: idempotent command submitted once
    Committing --> Completed: financial transaction and receipt succeeded
    Committing --> ReceiptRecovery: sale committed, receipt delivery incomplete
    Committing --> CommitFailed: no financial commit
    CommitFailed --> Tendering: safe retry token issued
    ReceiptRecovery --> Completed: receipt retried or alternative chosen
    Selling --> Suspended: hold, lock, or handover
    Suspended --> Selling: authorized resume with version check
    Selling --> ClosingShift
    ClosingShift --> Closed: counted cash and evidence committed
```

Each state must come from a server-authoritative read model and include allowed commands, denial reasons, stale/version information, safe-retry semantics, and a correlation ID. Client calculations may preview but never own financial truth.

Recommended page structure and annotated wireframe

STOQUIFY POS

Branch · Location · Terminal

Cashier

Shift

Net/Devices

[LOCK]

any unknown/stale state is explicit

[Help]

[EN]

[FR]

SELL

[Scan/Search_____]

[Category]

Exact scan feedback / last match / error

Quick keys / recent / favorites

Product price available

Product price available

Product price available

CART 4 items

Item A2 × XAF...

[-] [qty] [+] [Remove]

Item B1 × XAF...

SubtotalXAF...

TaxXAF...

TotalXAF...

[Customer / receipt options]

[REVIEW AND CHARGE – XAF...]

Readiness/exception strip: actionable blocker · safe retry · correlation ID

Tablet: 55/45 split with a collapsible catalog or cart, never sequential long scroll.
 Mobile: catalog body + sticky item/total/Review bar; cart and tender open as a full-height bottom sheet with preserved focus, clear close behavior, and no covered fields.
 Diagnostics: device detail, performance metrics, receipt history, and advanced controls live in secondary drawers, not in the repeated scan-to-charge path.

Annotations:

1. The top bar contains only custody and readiness facts that affect the next sale.
2. Scan/search feedback is immediate and differentiates no match, multiple match, not sellable, stale stock, and success.
3. Product tiles show one authoritative sellable quantity, price with currency, and concise eligibility state.
4. Cart total and the next allowed command remain visible at every supported viewport.
5. Review and Charge opens a method-aware tender state; it does not imply payment success.
6. Receipt recovery is a post-commit state and never invites the cashier to resubmit the sale.
7. Errors say what happened, whether money/stock changed, and what action is safe.

Required component and state contracts

Contract	Required fields/behavior
POS access context	organizationId, locationId, terminalId, cashierId, permissions, module entitlement, assignment, decision reason, evaluatedAt
Terminal readiness	device class, adapter ID, state, lastSeenAt, freshness, recoverable action; unknown is the default
Shift context	sessionId, owner, status, version, opening evidence, expected balance visibility policy, allowed commands, lock/handover state
Catalog item	itemId, barcode aliases, localized name, authoritative price/tax preview, sellableAvailable, inventoryVersion, eligibility/reason
Draft cart	cartId, version, owner/session scope, lines with stable IDs, server totals, blockers, expiresAt, lastCommandId
Mutation command	commandId, cartId, expectedVersion, command type, payload, client timestamp for diagnostics only
Tender capability	method, enabled, disabled reason, required fields, provider choices, overpay policy, reconciliation policy, entitlement
Commit result	saleId, immutable receipt number, financial state, payment state, inventory state, receipt state, safeRetry, correlationId
Exception approval	operation, actor, approver, reason code, bounded scope, expiresAt, fresh-auth evidence, result
Offline command	terminal sequence, idempotency key, signed actor/session context, provisional status, replay result, conflict reason

The client may cache and preview these structures, but action/service/database layers remain authoritative. React Query mutations should update only the affected aggregate and use versioned conflict responses instead of invalidating the entire POS namespace.

Required service and read-model changes

1. Create a POSSurfaceContext read model that resolves permission, module entitlement, organization/location/terminal assignment, shift custody, device capability, currency scale, and allowed tender capabilities in one result.
2. Split catalog search from exact barcode lookup; both return sellability and inventory version.
3. Add database-enforced draft identity and line uniqueness, cart versioning, and mutation idempotency.
4. Make currency scale authoritative in opening, tender, refund, and close schemas and persistence.
5. Replace the static tender list with configured method capabilities and required metadata.
6. Add a preflight command that returns authoritative totals, stock conflicts, credit checks, and approval requirements immediately before tender commit.
7. Define lock, suspend, handover, hold/resume, cancellation, refund, and void state transitions with auditable reason codes.
8. Define receipt outcome separately from financial outcome and expose retry/reprint/share commands without raw URLs.
9. Either remove the offline-readiness claim or connect every allowed offline command to a certified queue/replay contract.
10. Emit a shared correlation ID across action, service, database audit, payment provider, journal/outbox, and receipt delivery.

Accessibility and localization release requirements

- WCAG 2.2 AA is the target, not a certification claim from this audit.
- All icon buttons need programmatic names; toggles need selected/pressed state.
- Minimum target size is 44×44 CSS pixels in every mode, with no special toggle required.
- Cart changes, scan result, validation blocker, payment result, offline state, and receipt outcome need concise live announcements.
- Focus order follows sell → cart → tender; dialogs trap and restore focus; destructive confirmation identifies the exact item/operation.
- The full workflow must remain usable at 200% zoom and at 320 CSS pixels without two-dimensional scrolling.
- Color is supplemental; statuses include text/icon/pattern and meet contrast requirements in bright environments.
- Reduced-motion disables nonessential animation without suppressing progress/status.
- Every user-facing string, including device/offline/error copy, moves through the EN/FR catalogs.
- Money, dates, times, and quantities use organization locale/currency rules; XAF has zero fractional digits.
- French copy receives human review for accents, terminology, expansion, and operational brevity.

Performance and reliability budgets

These are proposed release budgets and must be measured against a production-like dataset, not inferred from unit tests.

Journey	Proposed budget
Local keystroke/touch feedback	p95 ≤100ms
Exact barcode lookup to acknowledged cart command, online	p95 ≤300ms, p99 ≤750ms
Text search first useful results	p95 ≤500ms after 150–250ms debounce
Cart mutation server acknowledgement	p95 ≤500ms
Tender preflight	p95 ≤750ms excluding external provider
Financial commit	p95 ≤2s excluding provider authorization; one visible in-flight command
Receipt outcome after commit	p95 ≤3s, otherwise explicit RETRY_REQUIRED without recharging
Initial interactive POS shell on store hardware	p75 ≤2.5s on the agreed network/device profile
Catalog DOM	Bounded/virtualized; no growth proportional to the full catalog
Refresh recovery	Authoritative shift/cart state restored ≤2s after shell readiness

Reliability gates: one idempotent outcome per command, no silent optimistic financial state, finite retry policy with jitter, explicit timeout semantics, degraded/offline truth, and no customer or payment secret in telemetry.

Observability contract

Emit privacy-minimized structured events for access decision, terminal readiness, shift open/lock/handover/close, exact scan, search, cart command, preflight conflict, approval, tender initiation/result, commit result, inventory conflict, receipt delivery/retry, queue/replay/conflict, and device transition.

Required common fields: correlationId, commandId where relevant, organization and location pseudonymous identifiers, terminal ID, workflow phase, outcome, safeRetry, latency, error taxonomy, app version, connectivity class, and device-adapter state. Exclude password, token, raw receipt URL, full customer contact, payment credentials, and free-text evidence unless explicitly redacted.

Dashboards and alerts should answer: which stores/terminals are affected, whether financial commit occurred, whether a retry is safe, whether failures cluster by provider/version/device, queue age/depth, and whether SLO/error-budget thresholds are breached.

Phased remediation roadmap

Phase 0 — Critical correctness, security, and transaction controls

1. Resolve tender truth: add Mobile Money provider metadata and remove/disable Store Credit until its service/ledger capability is enabled (POS-01).

2. Enforce currency-aware amount precision in schemas, services, inputs, variance, posting, and historical-data checks (POS-02).
3. Replace cashier substring permissions with an explicit allowlist and negative role assertions (POS-05).
4. Make permission, POS entitlement, assignment, and terminal access one shared enforced contract across sidebar, route, reads, and writes (POS-06).
5. Add draft-cart/line uniqueness, cart version, mutation idempotency, and PostgreSQL concurrency tests (POS-07).
6. Use authoritative sellable availability consistently and return explicit conflicts instead of silent clamping (POS-08).
7. Remove the offline-readiness claim until checkout is genuinely connected and certified (POS-03 containment).
8. Change device status to unknown/unavailable until real adapters report freshness (POS-04 containment).

Exit gate: no selectable impossible tender; no fractional XAF persistence; least-privilege cashier matrix passes; concurrent draft tests pass; every entry point enforces the same entitlement; UI contains no false offline/device readiness.

Phase 1 — High-impact workflow and usability

1. Introduce POSSurfaceContext and authoritative tender/preflight contracts.
2. Rebuild the persistent catalog/cart/charge hierarchy for desktop and tablet; add the sticky mobile review action (POS-10).
3. Implement or honestly disable park/resume and manager override; surface controlled refund/void/return entry points (POS-09).
4. Separate receipt recovery from financial retry and remove raw receipt URLs (POS-13).
5. Add inactivity lock, quick re-auth, suspend/handover, and recovery policy (POS-12).
6. Begin state-oriented component/service extraction behind unchanged command contracts (POS-15).

Exit gate: cashier completes all supported happy and exception paths without placeholder actions; cart/total/action are continuously reachable; lock/handover attribution is correct.

Phase 2 — Accessibility, responsive behavior, and robust states

1. Apply accessible names, state semantics, 44px targets, visible focus, live announcements, reduced motion, and zoom/reflow support (POS-11).
2. Certify EN/FR desktop/tablet/mobile layouts and operational copy; move offline/device strings into catalogs (POS-16).
3. Add explicit loading, stale, conflict, permission, partial-success, safe-retry, and service-unavailable components.
4. Simplify visual density and add a bright-environment theme/token profile (POS-17).
5. Minimize customer/receipt projections and verify role-safe masking (POS-13).

Exit gate: automated accessibility checks plus keyboard, screen-reader, zoom, touch, EN/FR, and responsive human review pass with linked evidence. This gate still does not self-certify WCAG compliance.

Phase 3 — Performance, resilience, offline, and operational polish

1. Add exact barcode lookup, debounced text search, cursor pagination/virtualization, targeted cache updates, image fallbacks, and visibility-aware polling (POS-14).
2. Implement the structured telemetry/correlation/SLO contract and fault-injection release evidence (POS-18).
3. Decide the offline product boundary. If in scope, implement encrypted queue, signed/idempotent commands, terminal sequencing, provisional receipt semantics, replay/conflict workbench, and retention controls (POS-03).
4. Build certified device adapters with test/fallback operations and fleet-level health (POS-04).
5. Run production-scale catalog, concurrent terminal, degraded network, refresh interruption, and provider failure tests.

Exit gate: proposed budgets are met on reference hardware/network/data; offline and device claims have evidence; release pipeline runs PostgreSQL and browser matrices.

Phase 4 — Optional evidence-supported differentiators

- Configurable quick-key layouts by store/category with governed deployment.
- Customer-facing display with privacy-safe totals and receipt consent.
- Role-safe sales recommendations that never change price, tax, stock, or tender truth.
- Device fleet operations dashboard and guided recovery playbooks.
- Evidence-backed replenishment hints linked to inventory provenance.

These are optional and must not displace Phases 0–3.

Dependencies, disagreements, and tradeoffs

Decision	Tension	Board position
Hide unsupported tenders vs show future capability	Sales demos may prefer visible breadth; cashiers require operational truth	Hide or visibly disable with reason. Never let an impossible method enter the commit path.
Strict available stock vs permissive carting	Strictness can interrupt selling when stock is stale; permissiveness delays failure	Use authoritative available quantity with explicit supervisor/offline policies. Never silently oversell.
Optimistic cart UI vs server authority	Optimism improves speed; stale state risks charge mismatch	Optimistic presentation is allowed only with versioned commands, rollback, and authoritative preflight. Financial totals remain server-owned.
Offline breadth vs control	Offline selling improves resilience; payments/inventory/accounting become distributed	Start with a narrow, policy-gated set of methods/items and explicit provisional state. Do not market offline before certification.
Rich dashboard vs high-throughput workflow	Managers value metrics; cashiers value focus	Keep sale-critical readiness persistent and move analytics/diagnostics to secondary surfaces.
Dark-only vs dual retail themes	Dark reduces glare in some stores; bright environments need higher luminance	Support tested light/high-contrast and dark profiles using the design system.

Decision	Tension	Board position
Monolith extraction vs urgent fixes	Refactor can delay controls; monolith increases regression risk	Make Phase 0 surgical under current contracts, then extract along workflow/state boundaries with characterization tests.
Module enforcement rollout	Immediate enforcement may strand customers configured under Sales	Reconcile package data first, provide an explicit migration window, then enforce one POS contract everywhere.

Prerequisite decisions: authoritative cashier role matrix; POS module/package policy; tender/provider configuration source; currency-scale policy; draft identity scope; stock reservation semantics; store hardware matrix; inactivity/handover policy; and offline product scope.

Verification results

Verification	Status	Result/evidence
Focused command: ProfessionalPOSSystem shift-close + pos.service + pos-shift-close	PASS	3 suites, 54 tests passed, 0 skipped; 96.816s
Expanded POS selection across page, components, hooks, actions, services, and lib	PASS WITH SKIPS	24 suites passed, 1 skipped; 199 tests passed, 7 skipped; 55.063s
TypeScript: npm run typecheck	PASS	Exit code 0
POS translation parity	PASS	EN 245 keys, FR 245 keys, zero missing; 16 identical values, mainly legitimate shared terms
Graphify architecture inspection	PASS	Relevant route/component/action/hook graphs and reports inspected; topology used with stale-line caveat
Current unauthenticated route behavior	PASS	/en/dashboard/pos redirected to /en/login before the browser session interruption
Current authenticated EN/FR desktop/tablet/mobile browser matrix	BLOCKED	Seeded login submission was authorized on the resumed run, but the fresh browser first reached localhost while the dev server was offline; subsequent navigation was blocked by the browser URL safety policy. The control was not bypassed.
Empty/populated cart, tender, search, permission, and error runtime states	BLOCKED	No persistent cart/shift/sale mutation was authorized; authenticated browser matrix could not continue
Opt-in PostgreSQL close certification	SKIPPED	Seven tests are intentionally gated on an opt-in PostgreSQL environment
Physical scanner/printer/drawer and offline certification	NOT RUN	Required hardware/network harness is not present in this audit

The earlier Phase 05 and Phase 09 UI/UX reports were reviewed. Their PASS WITH CONDITIONS outcomes concern robust-state/governance work and do not constitute POS browser, accessibility, payment-provider, offline, hardware, or release certification.

Unresolved blockers

1. Re-run the browser matrix in a fresh browser session after confirming the development server is already listening. Capture non-sensitive EN/FR views at 1440×1000, 1024×768, and 390×844.
2. Authorize a reversible, isolated audit-cart strategy if populated cart and tender screens must be runtime-inspected without leaving business records. The preferred implementation is an audited sandbox/test tenant or transaction rollback harness, not manual cleanup.
3. Run the opt-in PostgreSQL shift-close suite against an isolated disposable database and retain its immutable output.
4. Provide supported store hardware and network profiles for device, bright-environment, degraded-network, and offline validation.
5. Obtain specialist review before making WCAG, security, privacy, accounting, OHADA/SYSCOHADA, fiscal, PCI, or production-release certification claims.

Final decision and implementation handoff

The POS should not be represented as enterprise-grade until Phase 0 is complete and the missing browser/PostgreSQL evidence is closed. A controlled internal demo is reasonable if unsupported tenders, offline readiness, and synthetic hardware labels are clearly disclosed and no cashier seed is treated as a production role template.

The recommended implementation sequence is POS-01/02/05/06 first, POS-07/08 next, then truthful offline/device containment, followed by the workflow/responsive/accessibility work. This order protects financial and authorization truth before visual modernization.

Audit artifact changes are limited to this report and the evidence README. No application code, database contents, migration, seed data, design-system component, or persistent POS record was modified.